

ROB 550 Armlab Report G6 PM

Zheng Wang, Qizhang Shen, Divyanshu Singh Chauhan
 {zzww, qzshen, divsingh}@umich.edu

Abstract—This report explores computer vision techniques and robotic arm motion control within the ROB 550 Armlab project. Utilizing a LiDAR camera, we have developed a vision system that integrates with the Interbotix ReactorX-200 5-DOF robotic arm. Our system features automatic determination of intrinsic and extrinsic camera parameters and workspace rectification. By combining various OpenCV algorithms with clustering techniques, we achieve accurate real-time identification of object types, sizes, colors, and poses. For robotic arm motion control, our forward kinematics component employs the Denavit-Hartenberg (DH) table method, ensuring precise control of robot movements. We address the inverse kinematics problem by decomposing the joint variable set, enabling accurate gripper positioning and manipulation. Our path planning module generates a comprehensive list of the robot arm’s joint angles, optimizing efficiency during block picking and placing tasks. The effectiveness of our algorithms is validated through five distinct events, demonstrating the system’s robustness and versatility.

Keywords—Robotic arm control, Object detection, Path planning

I. INTRODUCTION

In the ROB 550 Armlab, we developed an autonomous system for a 5-DOF robot arm, using computer vision techniques, kinematics, and path planning to enable the robot to manipulate various objects.

For the acting part, we explored Forward and Inverse kinematics modeling of a manipulator, grasping, and 5-DOF rigid-body coordinate transforms using homogeneous coordinate transforms with Interbotix ReactorX-200 5-DOF Robot Arm. We implemented a click-to-pickup and drop interaction, as well as the ability to automatically pick up and place a block in a given position.

For the sensing part, we explored 3D image calibration, object detection with OpenCV, and usage of depth camera sensors with Intel RealSense LiDAR Camera L515. We developed a block detector that is able to detect various kinds of blocks and determine their center position, color, type, orientation, and so on.

For the reasoning part, we explored path planning, path smoothing, and state machines design. We designed different state machine functions for various tasks to build the autonomy of the robot arm.

Putting all these parts together, we designed the logic to complete several autonomous tasks and participated in the final competition achieving second place.

II. METHODOLOGY

A. Workspace Calibration

1) Intrinsic Camera Calibration

We found the intrinsic matrix for the Realsense camera using the ROS camera calibration package and a provided checkerboard. We performed the calibration for 4 times and computed the average intrinsic matrix.

2) Rough Extrinsic Camera Calibration

We used physical measurements of the lab apparatus and got a rough extrinsic matrix for the camera. We measured the displacement of the camera using a soft ruler and calculated the orientation using its 3-axis acceleration data. Finally, we used the extrinsic matrix and the camera intrinsic matrix to map image coordinates (u,v,d) to world coordinates.

3) Apriltag Calibration

To improve the accuracy of the calibration, Apriltags are used as fiducials. The tag family used is TagStandard41h12. The Apriltags’ detection information is acquired from the ROS topic *april-tag-msgs/msg/AprilTagDetectionArray*. The tag center locations are therefore obtained and drawn on the video frame to help ensure calibration quality. With the center location of each Apriltags acquired, the rotation and translation vectors *rvec* and **T** are obtained using the function *cv2.solvePnP()*. *rvec* is then transformed into the rotation matrix **R** using *cv2.Rodrigues()*[1]. Hence the extrinsic matrix **H** can be formed. The transformation from pixel coordinate to camera coordinate, and from camera to world coordinate is shown in the equations below.

$$\begin{bmatrix} X_c \\ Y_c \\ Z_c \end{bmatrix} = Z_c(u, v) \mathbf{K}^{-1} \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} \quad (1)$$

$$\begin{bmatrix} X_w \\ Y_w \\ Z_w \\ 1 \end{bmatrix} = \mathbf{H}^{-1} \begin{bmatrix} X_c \\ Y_c \\ Z_c \\ 1 \end{bmatrix} \quad (2)$$

Where **K** is the intrinsic matrix and **H** is the extrinsic matrix. The calibration will then be verified by reading the calculated coordinates of certain known points in the world coordinate system and comparing the result with their true locations. Such points include the Apriltags

and other points of selection on the grid. To facilitate this process, a grid of points corresponding to corners of the grid of the workspace is projected onto the image, followed by the removal of the scaling effect. This grid of points is converted using eq.1 and eq.2.

B. Block Detector

We developed a block detector that can detect the center, depth, orientation, contour, color, and type of blocks. It can also tell if the block is in a stack. The block detector employs a multi-step approach to identify and classify blocks in a given workspace. The system utilizes computer vision techniques and machine learning algorithms to process RGB and depth images from a camera. The key components of the methodology are as follows.

1) Image Preprocessing

We adopt two approaches to preprocess the RGB images and depth data from the camera. One is a height filter, which is to eliminate potential interference from the robot arm. The other is boundary masks, which is to provide the correct detection zone for different tasks.

For the height filter, we extract data from the depth image for specific depth intervals (850-1100mm for most tasks) and then process the RGB image, leaving only the parts where valid depth information exists.

For the boundary masks, we preset different boundary values to determine where to run the detector and where not to. Boundary values stored the pixel coordinates of the corner points of different regions, like whole workspace, arm station, positive half, lower left, and so on.

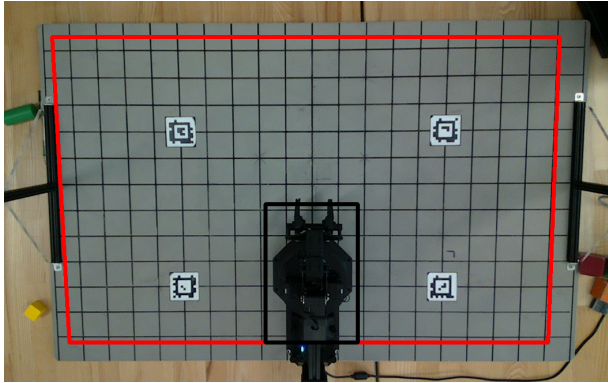


Fig. 1: Workspace Boundry

2) Block Detection

a) Contour Extraction

After preprocessing the image, we convert the RGB images to HSV format for contour extraction. This reduces the impact of lighting variations on edge detection and potentially improves the detection of color-based boundaries. First, we adapt an HSV range to filter the grey and white background. Then we use the function `cv2.findContours()` to find the raw contours in the binarized HSV image. For each contour we found, we used

the function `cv2.moments()` to calculate its moments and assigned a sub-region based on the area for subsequent operations. If the first moment of area ($M00$) calculated is too small (<200) or too large (>5000), then ignore this wrong contour. If the first moment of area ($M00$) is under 850, set the sub-region length to 50, else set it to 80.

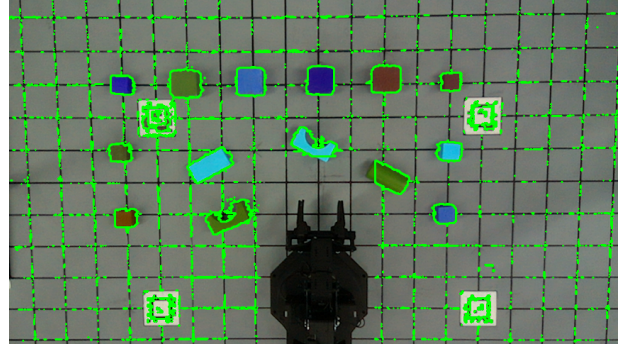


Fig. 2: Raw Contour of HSV Image

We captured the outlines of all objects, but these outlines are not precise, as they do not closely align with the edges of the top surfaces of the objects. This inaccuracy in contour alignment results in an imprecise calculation of the objects' center points, subsequently leading to failures in the mechanical arm's grasping attempts.

Based on the obtained initial contours, we compute their centroid positions and extract depth values from eight points surrounding the centroid. Taking the average of these depth values provides a relatively accurate depth measurement, enabling us to discern whether the objects are stacked. Due to the inherent limitations of depth image in achieving pixel-level precision, the contours obtained require further refinement.

b) Contour Optimization

After finding the raw contour and creating sub-regions for them, we can start to optimize the contour, which is to find the precise contour of the top surface of each block.

To make the detection more robust and accurate, we decided to combine RGB, LAB, and depth information and apply the K-means clustering algorithm to process each sub-region. We obtain the LAB color space representation for sub-regions and merge RGB, LAB, and depth map information, forming seven-dimensional feature vectors for each pixel coordinate.

We employ the K-means clustering algorithm[2] to cluster these multi-dimensional feature vectors, partitioning the image into four clusters. The most common cluster (cluster 0) is the background, and the second most common cluster (cluster 1) is the top surface of the block.

The rest clusters are lateral side or other disruptions that we can ignore.

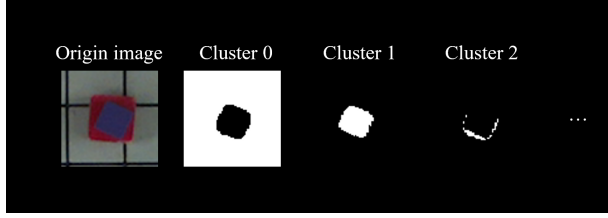


Fig. 3: Example of Clustering Results

Then we call the function `cv2.findContours()` again to find the precise contour for each second most common cluster of the sub-regions. With the precise contour, we can calculate the precise moments and precise centroid.

c) Block Classification

After gaining all the necessary information, we can classify the blocks. For each block, we have the following variables in Table I to define it.

TABLE I: Block classes

Variables	Definition
center	pixel coordinates of top surface centroid
depth	depth of centroid in camera frame
orientation	orientation of block in image
contour	contour of block in image
color	color of the block
type	type of the block
stack	if the block is stacked
world_xyz	world coordinates of the block

To find the center of a block, we call the function `cv2.moments()` to get the moments of the precise contour, and denote it as 'M'. Then $M['m10']/M['m00']$ gives the x-coordinate of the centroid while $M['m01']/M['m00']$ gives the y-coordinate.

The depth value at the precise center is retrieved from the depth image using the calculated coordinates.

To find the orientation of a block, we call the function `cv2.approxPolyDP()` to approximate the contour to a simple polygon. Then, we use `cv2.minAreaRect()` to get the minimum area rectangle that encloses this polygon. One return value of `cv2.minAreaRect()` is the box angle, which gives us the orientation of the block.

The contour is directly obtained from the `cv2.findContours()` function after refining the region using clustering.

To find the color of each block, we compare the feature vector used in clustering which contains RGB and LAB values with some reference vectors. Consider six colors: red, orange, yellow, green, blue, and purple.

We place nine blocks in the workspace for each color and gather their feature vector to calculate an average reference vector. To determine block color, we calculate the 2-norm distance between its feature vector and the references and set it to the nearest color.

We determine the type of a block by several 'if' judgments. First, we determine if it's a semi-circle by slicing the depth image. Starting from the top centroid, we make one slice at a 15mm height difference and another at a 5mm height difference. We then calculate the ratio of non-zero values in these two layers. If the ratio is bigger than 6, the block type is 'semi-circle'. If not, go to the next judgment.

Since we have approximated each contour to a simple polygon before, we have the number of edges and the width and height of the polygon. We always set the width to be the larger length. If the edge number is 3, the block type is 'triangle'. If the edge number is 5 and the width is bigger than 40, the block type is 'arch'. If the edge number is 5 and the width is not bigger than 40, the block type is 'up-semi-circle'. If the edge number is more than 5, the block type is 'cylinder'. If the edge number is 4, go to the next judgment.

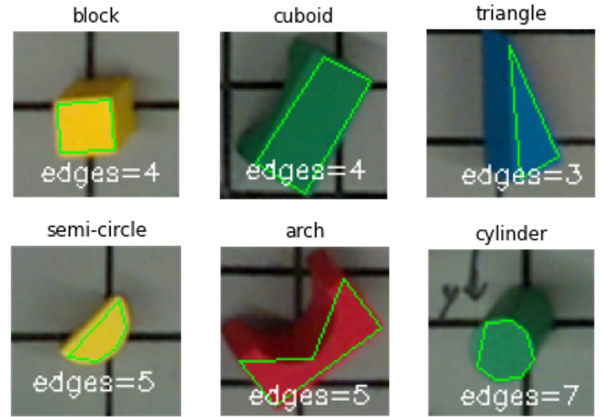


Fig. 4: Example of number of edges

If the edge number is 4, we calculate the ratio of width and height to distinguish square block and rectangle block. If the ratio is bigger than 1.8, the block type is 'rectangle'. If not, we can finally set the block type to square block, but we still need to distinguish between big and small blocks and if it's a stack. If the width is bigger than 30, the block type is 'big'. Else the block type is 'small'.

Finally, we calculate the depth gap between the centroid and the average depth of four corners of the sub-region. For a big block, if the depth gap is bigger than 50, set stack to True. For a small block, if the depth gap is bigger than 30, set stack to True. Otherwise, set stack to False.

To find world coordinates, we use intrinsic and extrinsic matrices to transform the center and depth from the camera frame to the world frame.

C. Kinematics

1) Forward Kinematics (FK)

Forward Kinematics in robotics refers to the calculation of the orientation and position of the end-effector which is the hand gripper in our case. In other words, it means to calculate the position (x, y, z) and orientation (ϕ, θ, ψ) given the joint angles of the 5 DOF robotic arm. We have used the ZYZ euler angle convention for our task.

We calculate this using the DH matrix or the Denavit-Hartenberg (DH) convention. A schematic of the arm with DH frames is shown in Fig.5 below.

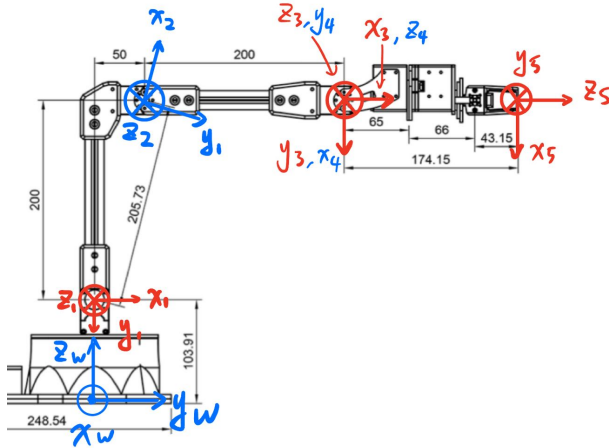


Fig. 5: Schematic of the arm with DH frames

Steps to calculate FK using DH matrix:

- 1) We define the DH frame for each joint in the robot. The DH frame for a joint has 4 elements named θ (joint angle), d (link offset), α (link length) and a (link twist). For each joint, these 4 elements are created and added to the matrix to create a final DH matrix as shown in table V.
- 2) Now, for each joint, we define a transformation matrix (4*4 matrix) using the calculated four DH parameters which accounts for both the translational and rotational shift from one joint to the next. The matrix is given as:

$$T_i = \begin{pmatrix} \cos \theta_i & -\sin \theta_i \cos \alpha_i & \sin \theta_i \sin \alpha_i & a_i \cos \theta_i \\ \sin \theta_i & \cos \theta_i \cos \alpha_i & -\cos \theta_i \sin \alpha_i & a_i \sin \theta_i \\ 0 & \sin \alpha_i & \cos \alpha_i & d_i \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

- 3) Now, we compute the one matrix transformation for each link starting from the base link to the last link. For each link, we substitute its respective DH parameters θ , d , α and a into the transformation matrix.

- 4) Now, we multiply all these calculated transformations which returns us a 4*4 final transformation matrix that encodes the position and orientation of the end-effector relative to the base frame.

The transformation matrix T is given by:

$$T = \begin{pmatrix} r_{11} & r_{12} & r_{13} & x \\ r_{21} & r_{22} & r_{23} & y \\ r_{31} & r_{32} & r_{33} & z \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

where,

$$\text{Position} = \begin{pmatrix} x \\ y \\ z \end{pmatrix}$$

and Rotation matrix is given by:

$$\text{Orientation} = \begin{pmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{pmatrix}$$

, therefore,

$$\phi = \text{acos}(r_{33})$$

$$\psi = \text{atan2}(r_{23}, r_{13})$$

$$\theta = \text{atan2}(r_{32}, -r_{31})$$

Thus, using these steps as part of the forward kinematics, we are able to retrieve the position and orientation of the end-effector.

2) Inverse Kinematics (IK)

Inverse Kinematics refers to the calculation of the configuration space when we are given the task space. In other words, our aim is to calculate the joint angles when we are given the end-effector position. There are two ways to solve the inverse kinematics:

- 1) 1. Geometrical Solution
- 2) 2. Solving System of Equations

We utilized the geometrical solution for solving our tasks of inverse kinematics.

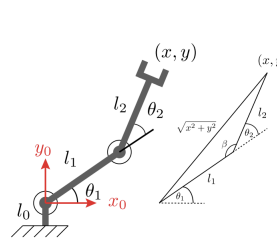


Fig. 6: 2-link mechanism

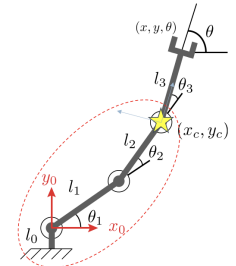


Fig. 7: 3-link mechanism

For a two link mechanism as shown in fig 6, we can easily calculate the coordinates of the joints if we are

given the end effector position using the trigonometric calculations. For the 3 link RRR mechanism as shown in fig 7, we can extend the results from this by considering the 3 link mechanism as a 2 link mechanism connected with a 3rd link. With this logic, we are able to fetch all the joint angles present in the mechanism.

Steps to calculate the IK:

- 1) We know the pose values which are the position (x, y, z) and orientation (θ, ϕ, ψ) of the end-effector.
- 2) From the given, (x, y, z) we get the (x_c, y_c, z_c) which is the position coordinates of the wrist joint.
- 3) Using the coordinates of the wrist joint calculated above and the link lengths, we get the joint angles of the elbow and shoulder joint.
- 4) The base joint angle can be calculated directly by calculating the angle it rotated in the x-y plane.
- 5) Finally, to calculate the wrist joint angle, we have the relation from trigonometry and offsetting between the joint angles and ϕ as:

$$\phi = \theta_{wrist} + \theta_{shoulder} + \theta_{elbow}$$

Thus, by following these steps, we were able to calculate all the joint angles given the end-effector position and orientation.

D. Robot Control

1) Teach and Repeat

Teach and Repeat method allows robots to perform simple, repetitive tasks by recording and replaying joint angles. When the user clicks a "Record" button in the GUI, the current joint angles and end effector state are saved in lists. To repeat the path, the "Repeat" button commands the robot to follow the recorded joint angles using `set_position()` and replicate end effector actions with `gripper.release()` or `gripper.grasp()`.

2) Click to Grab/Drop

Click to Grab/Drop allows the user to click on a point in the workspace. If the point is on a detected block, the arm will automatically pick it up. Otherwise, if the arm has already picked up a block, it will drop it at the selected position. This function provides a way to integrate computer vision and robot control and test their performance. The logic implemented is that when a point is clicked, the coordinate is inputted into the `inArea()` function to determine whether it lands on a block. If so, the center location and orientation of that block are obtained from the `self.camera.detection` class. Otherwise, the world coordinate of the selected point and an orientation of 0 is used for the following process.

Given the location and orientation, the `IK_geometric()` function is called to generate a series of waypoints forming a path from the current position of the arm's end effector to the pose to pick/place the block. Then `set_positions()` is called to carry out these waypoints, and

`gripper.grasp()` or `gripper.release()` is called depending on whether there is a block grasped in the end effector.

3) Auto Pick and Auto Place

the `auto_pick` and `auto_place` functions lay the foundations to complete the following tasks. Given the location of the block in the world frame, the system will use `IK_geometric` computation to determine whether the end effector can reach that position vertically. This process improves the accuracy of the pick-and-place actions. If the manipulator cannot reach the target position perfectly horizontally, the target location will be adjusted slightly closer, and the angle ϕ will be decreased. If the target remains unreachable after five such adjustments, the system will attempt a horizontal pick if the position is still within the workspace.

When picking or placing a block, the gripper initially moves to a position above the target and then slowly descends. For placing actions, a step-by-step dichotomy method is employed to lower the block incrementally while monitoring the torque variation in the motors. If the sum of the torques exceeds a defined threshold, it is assumed that the block has made contact with a surface, prompting the gripper to stop descending and release the block.

4) Competition 1: Sort 'n stack!

We use a color-based sorting algorithm, prioritizing blocks in a specific order (red, orange, yellow, green, blue, purple). We divide the task into two main phases: first handling stacks and red blocks (if exist), then placing all remaining blocks. The task begins by detecting blocks, unstacking any stacked blocks, and prioritizing red blocks for placement. We then proceed to sort and place the remaining blocks according to their size (big or small) and color. The system uses predefined placement positions for both big and small blocks, maintaining separate counters for each.

5) Competition 2: Line 'em up!

We defined specific positions for all 12 blocks of different types and colors. We start a loop to repeatedly detect blocks, use a color-based sorting algorithm to determine the order of processing and process them one by one. We place big and small square blocks in their predefined positions and ignore the distractor blocks. This loop continues until all six colors have been placed for both big and small blocks. After completing the main sorting, the system performs a final push operation to center the blocks.

6) Competition 3: To the sky!

We detect blocks in the lower left region of the workspace and enter a loop to pick up each detected block. After picking, we check if the gripper has successfully grasped a block. If successful, we proceed to place the block on a specific point (0, 255). We calculate the place height with the current block number. The

placement strategy adapts as the tower grows taller, using different placement techniques for various height ranges. We incorporate small offsets in x, y, and z coordinates to account for the increasing height and to maintain balance. When the block number reaches 8, we raise the arm up after each place action to avoid knocking down the stack. For very high placements (blocks 13 and 14), we use predefined joint angles to achieve precise positioning.

7) competition 4 and 5: Basketball Launcher

Our basketball launcher design is shown in Fig. 8 below. The spoon will first be pulled back and energy will be stored in the elastic band. The ball will then be loaded in the spoon. The mechanism can rotate about the pivot to aim and be fixed onto an acrylic plate by the slider when it is aimed. Finally one can push the trigger to launch the basketball.

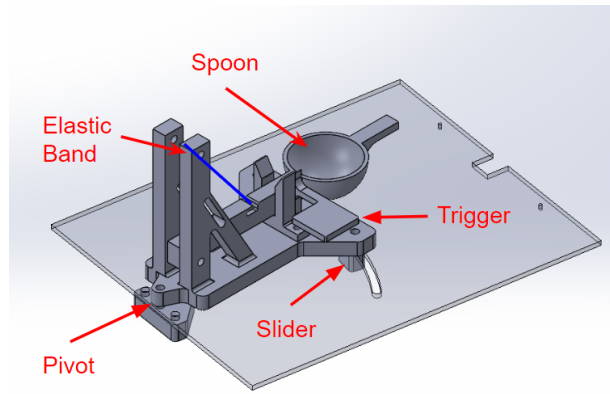


Fig. 8: the basketball launcher

III. RESULTS

A. Workspace Calibration

1) Intrinsic Camera Calibration

The average intrinsic matrices obtained from running ROS camera calibration package and from echoing the factory information are compared in Table.II below.

The two matrices are quite similar. The manual calibration shows slightly larger focal lengths, about 0.3% for f_x , and 0.8% for f_y . The factory calibration shows the principal point shifted more towards the center of the image compared to the manual calibration, about 1.8% for c_x , and 4.5% for c_y .

We trust the factory calibration more because the more times we calibrate manually, the closer the average result gets to the factory calibration. The factory calibration is likely performed in a controlled environment with precise equipment, which could make it more reliable. Potential sources of error include checkerboard quality, movement during calibration, lighting conditions, and the limited number of calibration images.

TABLE II: Intrinsic Matrices Comparison

Method	Intrinsic Matrix		
Manual Calibrated	899.1928	0.0000	648.7190
	0.0000	904.4954	365.1219
	0.0000	0.0000	1.0000
Factory	896.8611	0.0000	660.5231
	0.0000	897.2032	381.4194
	0.0000	0.0000	1.0000

2) Extrinsic Camera Calibration

The result of Apriltag detection is shown in Fig.14 in the appendices. By inspection, we conclude that the detection is accurate and hence supports the ASpriltag calibration. As shown in Table IV below, there is no significant difference between the hand measured and Apriltag calibrated extrinsic matrices in the rotation part, while the translational parts differs quite a lot. We determined that the source of error mainly comes from not aligning the measuring tap with the axes properly due to how the camera is mounted.

TABLE III: Extrinsic Matrices Comparison

Method	Extrinsic Matrix			
Hand Measured	0.999	0	-0.0274	48.0
	-0.00239	-0.996	-0.0871	197.0
	-0.0273	0.0872	-0.996	990.0
	0	0	0	1
Apriltag Calibrated	0.9997	-0.004736	-0.02508	35.72
	-0.002705	-0.9968	0.08040	167.1
	-0.02538	-0.08031	-0.9964	1005
	0	0	0	1

To verify the quality of the calibration, we hovered the cursor at the locations of (0, 175), (-300, -75) and (300, -75), and stacked from 2 to 6 blocks at these positions. We observed that the errors in the x and y axes is negligible and the error in the z axis is around $\pm 10mm$. We determined this error is mainly due to the nature of the PnP algorithm and was compensated by manually setting offsets. This nominal matrix is sufficient to find points in the world coordinate.

We use the four center points of detected April tags as the source points for perspective transformation. The destination points are (470, 560), (920, 560), (920, 290), (470, 290).

The values close to 1 on the diagonal of the rotation submatrix (0.9874, 0.9394, 1) suggest that there's min-

TABLE IV: Homography Matrix

Homography Matrix	$\begin{bmatrix} 0.9874 & -0.0634 & 2.2607 \\ 0.0114 & 0.9394 & 1.2928 \\ 0.00002 & -0.00009 & 1 \end{bmatrix}$

imal rotation between the camera and world coordinate systems. The translation values (2.2607, 1.2928) indicate a significant offset between the camera's origin and the world coordinate system's origin. The very small values in the third row (0.00002, -0.00009) suggest that the camera is almost perfectly aligned with the world coordinate system's z-axis, with minimal perspective effects.

We can see from Fig.9 that more than 95% of the grid points coincide with the intersection of the board.

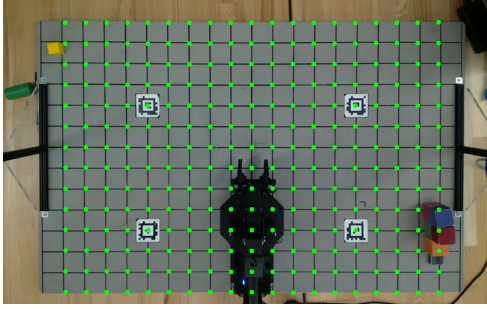


Fig. 9: Grid of points projected

B. Block Detector

When there are only square blocks, the detection results are shown in Fig.10.

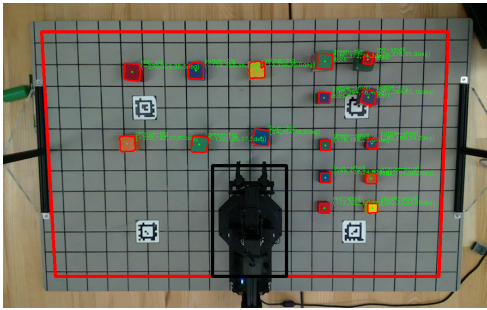


Fig. 10: Detection of square blocks

When there are various kinds of blocks, the detection results are shown in Fig.11.

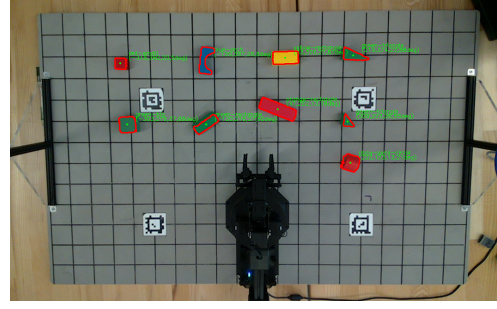


Fig. 11: Detection of various blocks

We can see that the detector can correctly detect block center, contour, orientation, color, type, and stack. To verify the accuracy of our block detector, we set a group of blocks as ground truth and visualize the uncertainty vs. location on the board using a heatmap in Fig.12. We get the uncertainty by calculating the Euclidean distance between the detected world coordinates and ground truth world coordinates. We can tell from the uncertainty heatmap that the uncertainty is always less than 1.

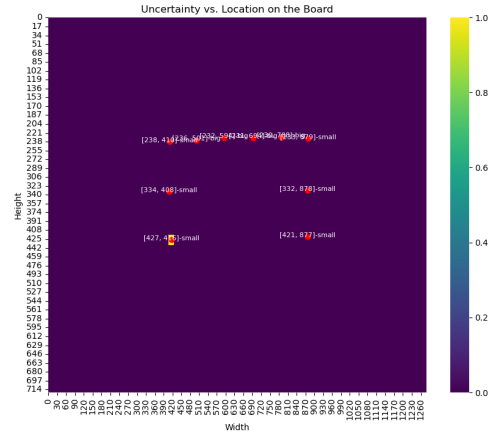


Fig. 12: Heatmap of uncertainty vs. location

C. Kinematics

1) Forward Kinematics

After implementation of the forward kinematics, we get the following DH table as shown in table V

TABLE V: DH Parameters Table

θ	d	a	α
$\theta_1 + \frac{\pi}{2}$	103.91	0	$-\frac{\pi}{2}$
$\theta_2 - c^*$	0	205.73	0
$\theta_3 + c^*$	0	200	0
$\theta_4 + \frac{\pi}{2}$	0	0	$\frac{\pi}{2}$
θ_5	174.15	0	0

The c^* shown in table accounts for the offset between the shoulder and the elbow joint of the arm and its value is calculated as:

$$\tan(c^*) = \frac{200}{50}$$

Simplified as:

$$c^* = \tan^{-1}(4) = 1.3258 \text{ radians}$$

2) Inverse Kinematics

Assuming, (x, y, z) are the world coordinates of the end-effector and ϕ is the angle of the link joining the wrist and the end-effector with the global horizontal plane. We define the various link lengths as $l_1 = 103.91\text{mm}$, $l_2 = 200\text{mm}$, $l_3 = 50\text{mm}$, $l_4 = 205.73\text{mm}$, $l_5 = 200\text{mm}$, $l_6 = 175\text{mm}$. We calculate the various joint angles as:

$$\alpha = \arctan\left(\frac{l_3}{l_2}\right), r = \sqrt{x_c^2 + y_c^2}, s = z_c - l_1$$

$$\theta_{elbow} = \frac{\pi}{2} + \alpha - \arccos\left(\frac{l_4^2 + l_5^2 - r^2 - s^2}{2l_4l_5}\right)$$

$$\theta_{base} = \arctan 2(-x, y)$$

D. Robot Control

1) teach and repeat

The robot arm is taught to swap two blocks between the two locations $(-100, 225)$ and $(100, 225)$ using $(250, 75)$ as an intermediate location. The robot arm is able to repeat the process for at least 10 times. For time saving considerations, the test was interrupted at the tenth cycle. The joint angles for each motor during one cycle of this task are recorded and shown in the figure below.

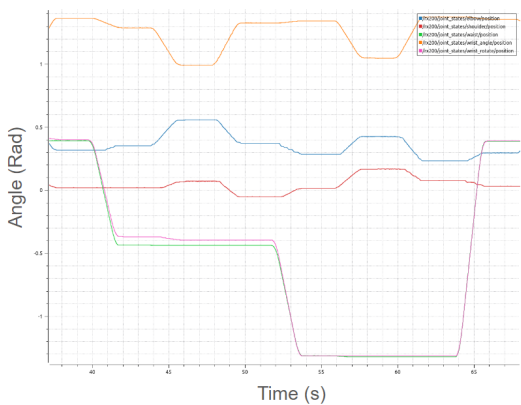


Fig. 13: Joint angles vs. Time during one cycle

2) Click to Grab/Drop

Our algorithm successfully controlled the arm to grab the selected block and place it at the designated location.

3) Auto Pick and Place

Both functions are incorporated into the constructions of the competition tasks and the performance of the functions are hence reflected.

4) competition 1: Sort 'n stack!

We chose level 3 for this event and got 300 points. Our finish time was 1 minute and 52 seconds.

5) competition 2: Line 'em up!

We chose level 3 for this event and got 458 points. The main influencing factor is the accuracy of auto pick. We failed to grab one of the big blocks. The pushing-to-center process is not stable so the line quality can't reach 5.

6) competition 3: To the sky!

We managed to stack 14 blocks.

7) competition 4 and 5: Basketball Launcher

Unfortunately, our basketball launcher was broken immediately before the competition during several empty shots. We believe this is because the excessive kinetic energy that should have been transferred to the ball caused a shock load that exceeds the material limits, since all of the parts are SLA 3D printed. Therefore, we were not able to officially evaluate the performance of our launcher, though there were several successful test shots with manual operation.

IV. DISCUSSION

During the competition, we observed that the arm has a low consistency when performing repetitive tasks such as picking up blocks from the same location. This can be improved by upgrading the mechanical design of the robot arm or the lower level control algorithm such as a better tuned PID control.

We also observed that when the end effector is approaching a block for pick up, it would occasionally follow a curved trajectory even though we have commanded several intermediate way points forming a straight line to facilitate this approaching. This can be improved by having a higher resolution of way points or implementing a numerical inverse kinematics instead, and specifying certain restrictions.

V. CONCLUSION

In this report, we presented an approach to camera calibration, including intrinsic calibration, rough extrinsic calibration, and automated extrinsic calibration using Apriltags. We detailed the transformation from pixel coordinates to the reconstructed workspace, and implemented a block detection and manipulation system. Additionally, we developed efficient path planning and state machine solutions, enabling the successful completion of competition tasks with this integrated robotic system. Future improvements could focus on upgrading the path planning algorithm such that the end effector is able to traverse the workspace following a certain trajectory or maintaining some orientation, and thereby improving the system's reliability and performance.

REFERENCES

- [1] J. E. Mebius, "Derivation of the euler-rodrigues formula for three-dimensional rotations from the general formula for four-dimensional rotations," 2007.
- [2] J. A. Hartigan and M. A. Wong, "Algorithm as 136: A k-means clustering algorithm," *Journal of the Royal Statistical Society. Series C (Applied Statistics)*, vol. 28, no. 1, pp. 100–108, 1979.
- [3] M. Spong, S. Hutchinson, and M. Vidyasagar, *Robot Modeling and Control*. Wiley, 2005. [Online]. Available: <https://books.google.com/books?id=wGapQAACAAJ>

VI. APPENDICES

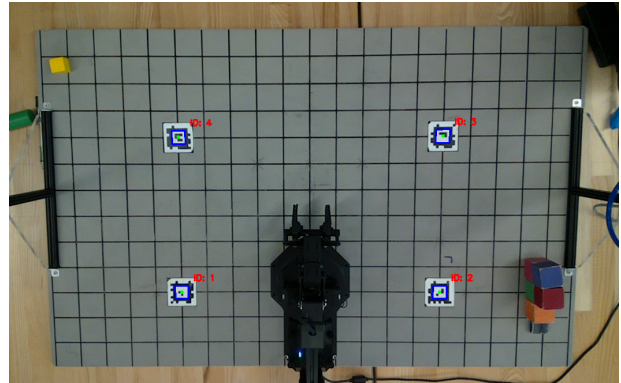


Fig. 14: Apriltag Detection